

One To One XML

(Advised to do it by your own)

One To One Annotation

Example 1:

Query11.java

```
package hibernatemapping;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToOne;
import javax.persistence.PrimaryKeyJoinColumn;

@Entity
public class Query11 {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @PrimaryKeyJoinColumn
    @Column(length = 10)
    private int id;
    @Column(length = 20)
    private String qname;

    @OneToOne(targetEntity=Response11.class, cascade=CascadeType.ALL)
    private Response11 responses;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getQname() {
        return qname;
    }
    public void setQname(String qname) {
        this.qname = qname;
    }
}
```

```

}
public Response11 getResponses() {
    return responses;
}
public void setResponses(Response11 responses) {
    this.responses = responses;
}
}

```

Response1.java

```

package hibernatemapping;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToOne;

//Cacheable
@Entity
public class Response11 {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(length = 10)
    private int id;

    @Column(length = 30)
    private String responsestr;

    @Column(length = 15)
    private String postedby;

    @OneToOne (targetEntity=Query11.class)
    private Query11 query;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getResponsestr() {
        return responsestr;
    }
}

```

```

public void setResponsestr(String responsestr) {
    this.responsestr = responsestr;
}
public String getPostedby() {
    return postedby;
}
public void setPostedby(String postedby) {
    this.postedby = postedby;
}
public Query11 getQuery() {
    return query;
}
public void setQuery(Query11 query) {
    this.query = query;
}
}

```

AppQR11.java

```

package hibernatemapping;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQR11 {
    public static void main(String[] args) {
        Response11 r1 = new Response11();
        r1.setResponsestr("Java is a programming Language");
        r1.setPostedby("Joe");

        Response11 r2 = new Response11();
        r2.setResponsestr("Servlet is an Interface");
        r2.setPostedby("Jas");

        Query11 query1 = new Query11();
        query1.setQname("What is JAVA?");
        query1.setResponses(r1);

        Query11 query2 = new Query11();
    }
}

```

```

        query2.setQname("What is SERVLET?");
        query2.setResponses(r2);

        r1.setQuery(query1);
        r2.setQuery(query2);

        Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(Query11.class)
                .addAnnotatedClass(Response11.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(conf.getProperties())
                .buildServiceRegistry();
        SessionFactory sf1 = conf.buildSessionFactory(sr);
        Session s1 = sf1.openSession();
        s1.beginTransaction();

        s1.save(query1);
        s1.save(query2);

        s1.getTransaction().commit();
    }
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.current_session_context_class">thread</property>
        <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>
        <property
name="hibernate.connection.password">oracle</property>
        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
roperty>
    
```

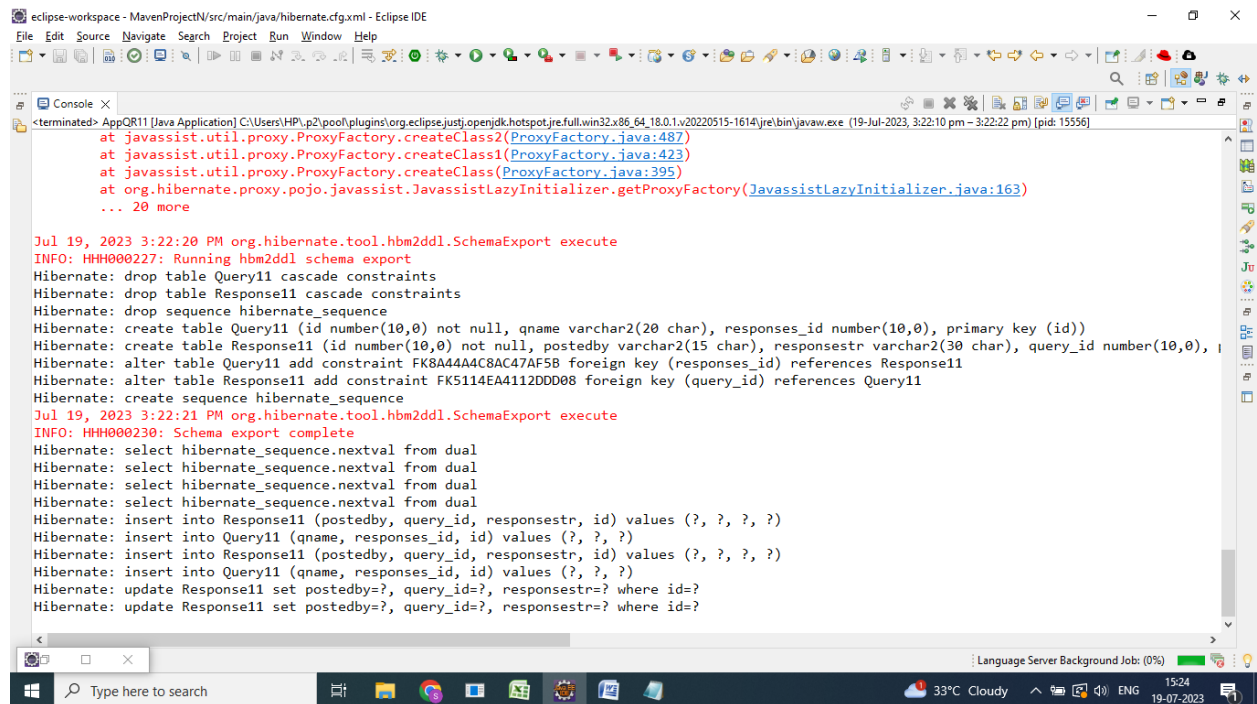
```

    <property
name="hibernate.connection.username">system</property>
    <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prop
erty>

    <property name="hbm2ddl.auto">create</property>
    <property name="show_sql">true</property>

</session-factory>
</hibernate-configuration>

```



The screenshot shows the Eclipse IDE interface. The top toolbar includes standard development tools like File, Edit, Source, Navigate, Search, Project, Run, Window, and Help. Below the toolbar is a console window titled 'Console' showing the output of a Java application. The logs indicate that the application is running hbm2ddl schema export, which involves creating and altering tables (Query11, Response11) and a sequence (hibernate_sequence). The logs also show the execution of SQL statements for inserting and updating data in the database.

```

<terminated> AppQR11 [Java Application] C:\Users\HP\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.18.0.1.v20220515-1614\jre\bin\javaw.exe (19-Jul-2023, 3:22:10 pm - 3:22:22 pm) [pid: 15556]
at javassist.util.proxy.ProxyFactory.createClass2(ProxyFactory.java:487)
at javassist.util.proxy.ProxyFactory.createClass1(ProxyFactory.java:423)
at javassist.util.proxy.ProxyFactory.createClass(ProxyFactory.java:395)
at org.hibernate.proxy.pojo.javassist.JavassistLazyInitializer.getProxyFactory(JavassistLazyInitializer.java:163)
... 20 more

Jul 19, 2023 3:22:20 PM org.hibernate.tool.hbm2ddl.SchemaExport execute
INFO: HHH000227: Running hbm2ddl schema export
Hibernate: drop table Query11 cascade constraints
Hibernate: drop table Response11 cascade constraints
Hibernate: drop sequence hibernate_sequence
Hibernate: create table Query11 (id number(10,0) not null, qname varchar2(20 char), responses_id number(10,0), primary key (id))
Hibernate: create table Response11 (id number(10,0) not null, postedby varchar2(15 char), responsestr varchar2(30 char), query_id number(10,0),
Hibernate: alter table Query11 add constraint FK8A44A4C8AC47AF5B foreign key (responses_id) references Response11
Hibernate: alter table Response11 add constraint FK5114EA4112DDD08 foreign key (query_id) references Query11
Hibernate: create sequence hibernate_sequence
Jul 19, 2023 3:22:21 PM org.hibernate.tool.hbm2ddl.SchemaExport execute
INFO: HHH000230: Schema export complete
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: select hibernate_sequence.nextval from dual
Hibernate: insert into Response11 (postedby, query_id, responsestr, id) values (?, ?, ?, ?)
Hibernate: insert into Query11 (qname, responses_id, id) values (?, ?, ?)
Hibernate: insert into Response11 (postedby, query_id, responsestr, id) values (?, ?, ?, ?)
Hibernate: insert into Query11 (qname, responses_id, id) values (?, ?, ?)
Hibernate: update Response11 set postedby=?, query_id=?, responsestr=? where id=?
Hibernate: update Response11 set postedby=?, query_id=?, responsestr=? where id=?

```

```
SQL Plus
SQL> select * from Query11;

-----
ID QNAME                RESPONSES_ID
-----
1 What is JAVA?          2
3 What is SERVLET?       4

SQL> select * from Response11;

-----
ID POSTEDBY      RESPONSESTR                QUERY_ID
-----
2 Joe            Java is a programming Language    1
4 Jas            Servlet is an Interface            3

SQL> █
```



AppQR11Fetch.java

```
package hibernatemapping;
```

```
import java.util.List;
```

```
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;
```

```
public class AppQR11Fetch {
    public static void main(String[] args) {
        Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(Query11.class)
                .addAnnotatedClass(Response11.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(conf.getProperties())
                .buildServiceRegistry();
```

```

SessionFactory sf1 = conf.buildSessionFactory(sr);
Session s1 = sf1.openSession();
s1.beginTransaction();

Query query = s1.createQuery("from Query11");

List<Query11> list = query.list();

for(Query11 q : list) {
    System.out.println("Query::");
    System.out.println(q.getId());
    System.out.println(q.getQname());
    Response11 res = q.getResponses();
    System.out.println("Response::");
    System.out.println(res.getId());
    System.out.println(res.getResponsestr());
    System.out.println(res.getPostedby());
}

s1.getTransaction().commit();
}
}

```

The screenshot shows the Eclipse IDE interface with the following details:

- Window Title:** eclipse-workspace - MavenProjectN\src\main\java\hibernatemapping\AppQR11Fetch.java - Eclipse IDE
- Menu Bar:** File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, Help
- Toolbar:** Standard Eclipse IDE toolbar with icons for file operations, editing, and running.
- Console Output:**

```

AppQR11Fetch [Java Application] C:\Users\HP\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.18.0.1.v20220515-1614\jre\bin\javaw.exe (19-Jul-2023, 3:25:25 pm - 3:25:36 pm) [pid: 20268]
at java.base/java.lang.reflect.Method.setAccessible(Method.java:194)
at javassist.util.proxy.SecurityActions.setAccessible(SecurityActions.java:103)
at javassist.util.proxy.FactoryHelper.toClass2(FactoryHelper.java:181)
at javassist.util.proxy.FactoryHelper.toClass(FactoryHelper.java:164)
at javassist.util.proxy.ProxyFactory.createClass3(ProxyFactory.java:502)
at javassist.util.proxy.ProxyFactory.createClass2(ProxyFactory.java:487)
at javassist.util.proxy.ProxyFactory.createClass1(ProxyFactory.java:423)
at javassist.util.proxy.ProxyFactory.createClass(ProxyFactory.java:395)
at org.hibernate.proxy.pojo.javassist.JavassistLazyInitializer.getProxyFactory(JavassistLazyInitializer.java:163)
... 20 more

Hibernate: select query11x0_.id as id0_, query11x0_.qname as qname0_, query11x0_.responses_id as responses3_0_ from Query11 query11x0_
Hibernate: select response11x0_.id as id1_2_, response11x0_.postedby as postedby1_2_, response11x0_.query_id as query4_1_2_, response11x0_.respoi
Query::
1
What is JAVA?
Response::
2
Java is a programming Language
Joe
Query::
3
What is SERVLET?
Response::
4
Servlet is an Interface
Jas

```
- System Tray:** Shows the date and time as 15:27 on 19-07-2023, along with weather information (34°C, Mostly cloudy) and system icons.

AppQR11Delete.java

```
package hibernatemapping;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppQR11Delete {
    public static void main(String[] args) {
        Configuration conf = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(Query11.class)
                .addAnnotatedClass(Response11.class);

        ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(conf.getProperties())
                .buildServiceRegistry();
        SessionFactory sf1 = conf.buildSessionFactory(sr);
        Session s1 = sf1.openSession();
        s1.beginTransaction();

        Object query = s1.get(Query11.class, 1);

        if(query != null) {
            s1.delete(query);
            System.out.println("Query deleted");
        }
        s1.getTransaction().commit();
    }
}
```

In **hibernate.cfg.xml** file, ensure to update

```
<property name="hbm2ddl.auto"> </property>
```



```
SQL Plus
4 Jas          Servlet is an Interface          3

SQL> select * from Query11;

-----
ID QNAME          RESPONSES_ID
-----
1 What is JAVA?          2
3 What is SERVLET?      4

SQL> select * from Response11;

-----
ID POSTEDBY      RESPONSESTR          QUERY_ID
-----
2 Joe          Java is a programming Language      1
4 Jas          Servlet is an Interface          3

SQL> select * from Query11;

-----
ID QNAME          RESPONSES_ID
-----
3 What is SERVLET?      4

SQL> select * from Response11;

-----
ID POSTEDBY      RESPONSESTR          QUERY_ID
-----
4 Jas          Servlet is an Interface          3

SQL>
```

Example 2:

Step 1: Set up the Project

1. Create a new Java project in Eclipse.
2. Add the Hibernate library to your project's classpath.
3. Add the database driver library (e.g., MySQL JDBC driver) to the classpath.

Step 2: Define the Entity Class

1. Create a new Java class called "Student" in your project.
2. Annotate the class with `@Entity` to mark it as a persistent entity.
3. Add the `@Table` annotation to specify the corresponding database table name.
4. Add the `@Id` annotation to denote the primary key field.
5. Annotate other fields with appropriate annotations (e.g., `@Column`, `@GeneratedValue`) to specify their mapping to database columns.
6. Add the `@PrimaryKeyJoinColumn` annotation.
7. Add the `@OneToOne(cascade = CascadeType.ALL, mappedBy = "product")` annotation.

Step 1:

Product.java

```
package com.company.hibernateonetoone;

import javax.persistence.CascadeType;
import javax.persistence.Column;
```

```

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.OneToOne;
import javax.persistence.PrimaryKeyJoinColumn;
import javax.persistence.Table;

@Entity
@Table(name = "PRODUCT")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    @PrimaryKeyJoinColumn
    @Column(length = 10)
    private int productId;
    @Column(length = 10)
    private String name;
    @Column(length = 15)
    private String description;
    @Column(length = 10)
    private float price;

    @OneToOne(cascade = CascadeType.ALL, mappedBy = "product")
    private ProductDetail productDetail;

    public Product() {
    }

    public int getProductId() {
        return productId;
    }

    public ProductDetail getProductDetail() {
        return productDetail;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }
}

```

```

    }

    public float getPrice() {
        return price;
    }

    public void setProductId(int productId) {
        this.productId = productId;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public void setPrice(float price) {
        this.price = price;
    }

    public void setProductDetail(ProductDetail productDetail) {
        this.productDetail = productDetail;
    }
}

```

ProductDetail.java

```

package com.company.hibernateonetoone;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.MapId;
import javax.persistence.OneToOne;
import javax.persistence.Table;

@Entity
@Table(name = "PRODUCT_DETAIL")
public class ProductDetail {
    @Id
    @Column(length = 5)

```

```

        private int productId;
@Column (length = 10)
        private String partNumber;
@Column(length = 10)
        private float weight;

@OneToOne
@JoinColumn(name = "productId")
@MapsId
        private Product product;

    public ProductDetail() {
    }

    public int getProductId() {
        return productId;
    }
    public String getPartNumber() {
        return partNumber;
    }

    public Product getProduct() {
        return product;
    }

    public float getWeight() {
        return weight;
    }

    public void setProductId(int productId) {
        this.productId = productId;
    }

    public void setPartNumber(String partNumber) {
        this.partNumber = partNumber;
    }

    public void setWeight(float weight) {
        this.weight = weight;
    }

    public void setProduct(Product product) {
        this.product = product;
    }
}

```

Step 2:

PPDManager.java

```
package com.company.hibernateonetoone;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class PPDManager {
    static Session getSession() {
        Configuration config = new Configuration().configure();

        ServiceRegistry sr = new ServiceRegistryBuilder()
            .applySettings(config.getProperties())
            .buildServiceRegistry();
        SessionFactory factory = config.buildSessionFactory(sr);

        Session session = factory.openSession();
        return session;
    }
    public static void main(String[] args) {

        Session session = getSession();
        session.beginTransaction();

        Product product = new Product();
        product.setName("Mouse");
        product.setDescription("Dell");
        product.setPrice(700);

        // creates product detail
        ProductDetail detail = new ProductDetail();
        detail.setPartNumber("qwerty123");
        detail.setWeight(1);

        product.setProductDetail(detail);
        detail.setProduct(product);

        // persists the product
        session.save(product);
    }
}
```

```

        session.save(detail);

    session.getTransaction().commit();
}
}

```

Step 3: Configure Hibernate

1. Create a Hibernate configuration file (e.g., `hibernate.cfg.xml`) in your project's classpath (right click on **src/main/java** - **new** - **file** - specify the file name (e.g. hibernate.cfg.xml) - **Finish**).
2. Configure the database connection details, dialect, and other properties in the configuration file. For example:

Hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property
name="hibernate.connection.password">oracle</property>
        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
        <property
name="hibernate.connection.username">system</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="show_sql">true</property>

        <mapping class="com.company.hibernateonetoone.Product"/>
        <mapping class="com.company.hibernateonetoone.ProductDetail"/>
    </session-factory>
</hibernate-configuration>

```